

Ejercicios de Métodos Numéricos

Capítulo 2: Búsqueda de Raíces

Ejercicios 2.51 – 2.61

Métodos Numéricos y Análisis

15 de febrero de 2026

Índice

1. Métodos de Búsqueda de Raíces	3
1.1. Ejercicio 2.51: Método de Bisección	3
1.1.1. Desarrollo Teórico	3
1.2. Ejercicio 2.52: Método de Regula Falsi	4
1.2.1. Desarrollo Teórico	4
1.3. Ejercicio 2.53: Método de Newton	5
1.3.1. Desarrollo Teórico	5
1.4. Ejercicio 2.54: Método de la Secante	7
1.4.1. Desarrollo Teórico	7
1.5. Ejercicio 2.55: Iteraciones Necesarias para Bisección	8
1.5.1. Solución	8
1.6. Ejercicio 2.56: Análisis de $x^3 - 3 = 0$	10
1.6.1. Planteamiento del Problema	10
1.7. Ejercicio 2.57: Demostración de Códigos de Búsqueda de Raíces	12
1.7.1. Reformulación del Problema	12
1.8. Ejercicio 2.58: Análisis de Orden de Convergencia	14
1.8.1. Parte (a): Transformación logarítmica	14
1.8.2. Parte (b): Pendiente en gráfica log-log	14
1.8.3. Parte (c): Determinación del orden de convergencia	15
1.8.4. Parte (d): Interpretación del orden de convergencia	15
1.9. Ejercicio 2.59: Predicción de Errores con Newton	17
1.9.1. Solución	17
1.10. Ejercicio 2.60: Método de Taylor de Orden N	19
1.10.1. Parte (a): Derivación del Método de Taylor de Orden 2	19
1.10.2. Parte (b): Demostración del Código	22
1.10.3. Parte (c): Gráficas de Orden de Convergencia	23
1.10.4. Parte (d): Pros y Contras del Método	25
1.11. Ejercicio 2.61: Caída de Objeto con Resistencia del Aire	26
1.11.1. Parte (a): Unidades del parámetro k	27
1.11.2. Parte (b): Tiempo de caída	28
1.11.3. Parte (c): Análisis de Sensibilidad	29
2. Solucionario - Gráficas	31
2.1. Ejercicio 2.51: Método de Bisección	31
2.2. Ejercicio 2.52: Método de Regula Falsi	32
2.3. Ejercicio 2.53: Método de Newton	33
2.4. Ejercicio 2.54: Método de la Secante	35
2.5. Ejercicio 2.55: Iteraciones Necesarias para Bisección	37
2.6. Ejercicio 2.56: Análisis de $x^3 - 3 = 0$	38
2.7. Ejercicio 2.57: Demostración de Códigos de Búsqueda de Raíces	39
2.8. Ejercicio 2.58: Análisis de Orden de Convergencia	40
2.9. Ejercicio 2.59: Predicción de Errores con Newton	41
2.10. Ejercicio 2.60: Método de Taylor de Orden N	41

2.11. Ejercicio 2.61: Caída de Objeto con Resistencia del Aire	42
3. Conclusiones	43

1. Métodos de Búsqueda de Raíces

En esta sección se presentan ejercicios relacionados con los métodos numéricos fundamentales para la aproximación de raíces de funciones continuas y diferenciables.

1.1. Ejercicio 2.51: Método de Bisección

Enunciado:

Sea $f(x)$ una función continua en el intervalo $[a, b]$ donde $f(a) \cdot f(b) < 0$. Proporcione claramente todos los detalles matemáticos de cómo el Método de Bisección aproxima la raíz de la función $f(x)$ en el intervalo $[a, b]$.

1.1.1. Desarrollo Teórico

Fundamento Matemático:

El Método de Bisección se basa en el Teorema del Valor Intermedio, el cual establece que si $f(x)$ es continua en $[a, b]$ y $f(a) \cdot f(b) < 0$, entonces existe al menos un valor $c \in (a, b)$ tal que $f(c) = 0$.

Algoritmo:

1. **Inicialización:** Dados $a_0 = a$ y $b_0 = b$ tales que $f(a_0) \cdot f(b_0) < 0$.

2. **Iteración k :** Para $k = 0, 1, 2, \dots$

- Calcular el punto medio:

$$c_k = \frac{a_k + b_k}{2}$$

- Evaluar $f(c_k)$

▪ Criterio de decisión:

- Si $f(c_k) = 0$, entonces c_k es la raíz exacta y el algoritmo termina.
- Si $f(a_k) \cdot f(c_k) < 0$, entonces la raíz está en $[a_k, c_k]$:

$$a_{k+1} = a_k, \quad b_{k+1} = c_k$$

- Si $f(c_k) \cdot f(b_k) < 0$, entonces la raíz está en $[c_k, b_k]$:

$$a_{k+1} = c_k, \quad b_{k+1} = b_k$$

3. **Criterio de paro:** El algoritmo se detiene cuando se cumple una de las siguientes condiciones:

- $|b_k - a_k| < \varepsilon$ (tolerancia en el intervalo)
- $|f(c_k)| < \delta$ (tolerancia en la función)
- Se alcanza un número máximo de iteraciones

Propiedades:

- **Convergencia garantizada:** Siempre converge bajo las condiciones iniciales establecidas.
- **Tasa de convergencia:** Lineal con razón $1/2$. El error se reduce a la mitad en cada iteración:

$$\text{Error}_k \leq \frac{b_0 - a_0}{2^k}$$

- **Robustez:** No requiere el cálculo de derivadas.
- **Limitación:** Convergencia relativamente lenta comparada con métodos de orden superior.

1.2. Ejercicio 2.52: Método de Regula Falsi

Enunciado:

Sea $f(x)$ una función continua en el intervalo $[a, b]$ donde $f(a) \cdot f(b) < 0$. Proporcione claramente todos los detalles matemáticos de cómo el Método de Regula Falsi (Falsa Posición) aproxima la raíz de la función $f(x)$ en el intervalo $[a, b]$.

1.2.1. Desarrollo Teórico

Fundamento Matemático:

El Método de Regula Falsi mejora el Método de Bisección al utilizar interpolación lineal en lugar de simplemente dividir el intervalo por la mitad. La idea es aproximar la función por una línea recta que conecta los puntos $(a, f(a))$ y $(b, f(b))$, y encontrar donde esta línea cruza el eje x .

Algoritmo:

1. **Inicialización:** Dados $a_0 = a$ y $b_0 = b$ tales que $f(a_0) \cdot f(b_0) < 0$.
2. **Iteración k :** Para $k = 0, 1, 2, \dots$
 - Calcular el punto de intersección de la secante:

$$c_k = b_k - f(b_k) \cdot \frac{b_k - a_k}{f(b_k) - f(a_k)}$$

Equivalentemente:

$$c_k = \frac{a_k f(b_k) - b_k f(a_k)}{f(b_k) - f(a_k)}$$

- Evaluar $f(c_k)$
- **Criterio de decisión:**
 - Si $f(c_k) = 0$, entonces c_k es la raíz exacta.
 - Si $f(a_k) \cdot f(c_k) < 0$, entonces:

$$a_{k+1} = a_k, \quad b_{k+1} = c_k$$

- Si $f(c_k) \cdot f(b_k) < 0$, entonces:

$$a_{k+1} = c_k, \quad b_{k+1} = b_k$$

3. **Criterio de paro:** Similar al Método de Bisección.

Propiedades:

- **Convergencia:** Garantizada bajo las condiciones iniciales, pero puede ser más lenta que bisección en casos patológicos.
- **Tasa de convergencia:** Generalmente superlineal, pero puede degradarse a sublineal si uno de los extremos permanece fijo.
- **Ventaja:** Utiliza información de la pendiente de la función, lo que típicamente resulta en menos iteraciones que bisección.
- **Desventaja:** Puede quedarse .atascado.en un extremo del intervalo si la función tiene curvatura pronunciada.

1.3. Ejercicio 2.53: Método de Newton

Enunciado:

Sea $f(x)$ una función diferenciable con una raíz cerca de $x = x_0$. Proporcione claramente todos los detalles matemáticos de cómo el Método de Newton aproxima la raíz de la función $f(x)$.

1.3.1. Desarrollo Teórico

Fundamento Matemático:

El Método de Newton (también conocido como Método de Newton-Raphson) se basa en la aproximación lineal de la función mediante su serie de Taylor de primer orden. La idea es que cerca de x_k , la función se puede aproximar por:

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k)$$

Para encontrar la raíz de esta aproximación lineal, igualamos a cero y despejamos x :

$$0 = f(x_k) + f'(x_k)(x - x_k)$$

$$x = x_k - \frac{f(x_k)}{f'(x_k)}$$

Algoritmo:

1. **Inicialización:** Dado un punto inicial x_0 cerca de la raíz.

2. **Fórmula iterativa:** Para $k = 0, 1, 2, \dots$

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

siempre que $f'(x_k) \neq 0$.

3. **Criterio de paro:**

- $|x_{k+1} - x_k| < \varepsilon$ (cambio relativo pequeño)
- $|f(x_{k+1})| < \delta$ (valor de función pequeño)
- Número máximo de iteraciones alcanzado

Interpretación Geométrica:

En cada iteración, se traza la recta tangente a la curva $y = f(x)$ en el punto $(x_k, f(x_k))$. La siguiente aproximación x_{k+1} es el punto donde esta tangente cruza el eje x .

Propiedades:

- **Convergencia cuadrática:** Cuando x_0 está suficientemente cerca de la raíz x^* y $f'(x^*) \neq 0$, el método tiene convergencia cuadrática:

$$|x_{k+1} - x^*| \leq C|x_k - x^*|^2$$

Esto significa que el número de dígitos correctos aproximadamente se duplica en cada iteración.

- **Requisito de derivada:** Necesita evaluar tanto $f(x)$ como $f'(x)$ en cada iteración.
- **Sensibilidad a la inicialización:** Puede diverger o converger a una raíz incorrecta si x_0 no está suficientemente cerca de la raíz deseada.
- **Problemas potenciales:**
 - Falla si $f'(x_k) = 0$ o es muy pequeño
 - Puede oscilar entre dos valores
 - Puede divergir si la función tiene comportamiento patológico

Análisis de Convergencia:

Sea x^* la raíz exacta y $e_k = x_k - x^*$ el error en la iteración k . Expandiendo $f(x_k)$ en serie de Taylor alrededor de x^* :

$$f(x_k) = f(x^*) + f'(x^*)(x_k - x^*) + \frac{f''(\xi)}{2}(x_k - x^*)^2$$

donde ξ está entre x_k y x^* . Como $f(x^*) = 0$:

$$f(x_k) = f'(x^*)e_k + \frac{f''(\xi)}{2}e_k^2$$

El error en la siguiente iteración es:

$$e_{k+1} = e_k - \frac{f(x_k)}{f'(x_k)} \approx e_k - \frac{f'(x^*)e_k + \frac{f''(x^*)}{2}e_k^2}{f'(x^*)} = -\frac{f''(x^*)}{2f'(x^*)}e_k^2$$

Por lo tanto:

$$|e_{k+1}| \approx \frac{|f''(x^*)|}{2|f'(x^*)|}|e_k|^2$$

confirmando la convergencia cuadrática con $C = \frac{|f''(x^*)|}{2|f'(x^*)|}$.

1.4. Ejercicio 2.54: Método de la Secante

Enunciado:

Sea $f(x)$ una función continua con una raíz cerca de $x = x_0$. Proporcione claramente todos los detalles matemáticos de cómo el Método de la Secante aproxima la raíz de la función $f(x)$.

1.4.1. Desarrollo Teórico

Fundamento Matemático:

El Método de la Secante es una modificación del Método de Newton que evita el cálculo de la derivada. En lugar de usar la derivada exacta $f'(x_k)$, se aproxima mediante una diferencia finita usando los dos puntos más recientes:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

Algoritmo:

1. **Inicialización:** Dados dos puntos iniciales x_0 y x_1 cerca de la raíz.
2. **Fórmula iterativa:** Para $k = 1, 2, 3, \dots$

$$x_{k+1} = x_k - f(x_k) \cdot \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$

Equivalentemente:

$$x_{k+1} = \frac{x_{k-1}f(x_k) - x_kf(x_{k-1})}{f(x_k) - f(x_{k-1})}$$

3. **Criterio de paro:** Similar a los métodos anteriores.

Interpretación Geométrica:

En cada iteración, se traza una línea secante que pasa por los puntos $(x_{k-1}, f(x_{k-1}))$ y $(x_k, f(x_k))$. La siguiente aproximación x_{k+1} es el punto donde esta secante cruza el eje x .

Propiedades:

- **Orden de convergencia:** Superlineal con orden $\phi = \frac{1+\sqrt{5}}{2} \approx 1,618$ (número áureo):

$$|x_{k+1} - x^*| \leq C|x_k - x^*|^\phi$$

- **Ventaja sobre Newton:** No requiere el cálculo de derivadas, solo evaluaciones de la función.
- **Desventaja respecto a Newton:** Convergencia más lenta (orden 1.618 vs. 2).
- **Ventaja sobre Bisección/Regula Falsi:** No requiere que los puntos iniciales encierren la raíz.
- **Requisito de memoria:** Necesita almacenar dos puntos previos en lugar de uno.
- **Problemas potenciales:**
 - Falla si $f(x_k) = f(x_{k-1})$ (denominador cero)
 - Puede diverger si los puntos iniciales no están suficientemente cerca de la raíz

Comparación con el Método de Newton:

Cuadro 1: Comparación: Método de Newton vs. Método de la Secante

Característica	Newton	Secante
Orden de convergencia	2 (cuadrática)	$\approx 1,618$ (superlineal)
Evaluaciones por iteración	f y f'	Solo f
Puntos iniciales requeridos	1	2
Costo computacional	Mayor	Menor

1.5. Ejercicio 2.55: Iteraciones Necesarias para Bisección

Enunciado:

¿Cuántas iteraciones del método de bisección son necesarias para aproximar $\sqrt{3}$ dentro de 10^{-3} , 10^{-4} , $\dots$, 10^{-15} usando el intervalo inicial $[a, b] = [0, 2]$? Ver Teorema 2.2.

1.5.1. Solución

Teorema 2.2 (Cota de Error para Bisección):

Después de n iteraciones del método de bisección, el error absoluto está acotado por:

$$|x_n - x^*| \leq \frac{b_0 - a_0}{2^{n+1}}$$

donde $[a_0, b_0]$ es el intervalo inicial y x^* es la raíz exacta.

Problema Específico:

Queremos encontrar $\sqrt{3}$, que es la raíz de $f(x) = x^2 - 3$ en el intervalo $[0, 2]$.

- $a_0 = 0$
- $b_0 = 2$
- $b_0 - a_0 = 2$

Para garantizar un error menor que ε , necesitamos:

$$\frac{2}{2^{n+1}} < \varepsilon$$

Despejando n :

$$2^{n+1} > \frac{2}{\varepsilon}$$

$$n + 1 > \log_2 \left(\frac{2}{\varepsilon} \right)$$

$$n > \log_2 \left(\frac{2}{\varepsilon} \right) - 1$$

Como n debe ser entero, tomamos:

$$n = \left\lceil \log_2 \left(\frac{2}{\varepsilon} \right) - 1 \right\rceil$$

Cálculos para cada tolerancia:

Cuadro 2: Iteraciones necesarias para diferentes tolerancias

Tolerancia ε	$\log_2(2/\varepsilon) - 1$	Iteraciones n
10^{-3}	9.97	10
10^{-4}	13.29	14
10^{-5}	16.61	17
10^{-6}	19.93	20
10^{-7}	23.25	24
10^{-8}	26.58	27
10^{-9}	29.90	30
10^{-10}	33.22	34
10^{-11}	36.54	37
10^{-12}	39.86	40
10^{-13}	43.18	44
10^{-14}	46.51	47
10^{-15}	49.83	50

Fórmula general usando cambio de base:

$$n = \left\lceil \frac{\log(2/\varepsilon)}{\log(2)} - 1 \right\rceil = \lceil \log_2(2) - \log_2(\varepsilon) - 1 \rceil = \lceil -\log_2(\varepsilon) \rceil$$

Para $\varepsilon = 10^{-k}$:

$$n = \lceil -\log_2(10^{-k}) \rceil = \lceil k \log_2(10) \rceil \approx \lceil 3,32k \rceil$$

Observación:

Se requieren aproximadamente 3.32 iteraciones adicionales por cada dígito decimal adicional de precisión.

1.6. Ejercicio 2.56: Análisis de $x^3 - 3 = 0$

Enunciado:

Refiérase al Ejemplo 2.1 y demuestre que obtiene los mismos resultados al resolver el problema $x^3 - 3 = 0$. Genere versiones de todas las gráficas del Ejemplo y proporcione descripciones exhaustivas de lo que aprende de cada gráfica.

1.6.1. Planteamiento del Problema

Queremos encontrar la raíz de:

$$f(x) = x^3 - 3 = 0$$

Es decir, queremos encontrar $x = \sqrt[3]{3} \approx 1,44224957\dots$

Análisis Preliminar:

- La función $f(x) = x^3 - 3$ es continua y diferenciable en todo $\mathbb{R}$.
- $f(0) = -3 < 0$ y $f(2) = 5 > 0$, por lo que existe una raíz en $(0, 2)$ por el Teorema del Valor Intermedio.
- $f'(x) = 3x^2 > 0$ para $x \neq 0$, por lo que f es estrictamente creciente.
- La raíz en $(0, 2)$ es única.

Gráficas a Generar:

1. **Gráfica de la función:** Visualización de $f(x) = x^3 - 3$ mostrando el cruce con el eje x .
2. **Iteraciones de Bisección:** Visualización del estrechamiento del intervalo en cada iteración.
3. **Iteraciones de Newton:** Visualización de las tangentes sucesivas que convergen a la raíz.
4. **Iteraciones de la Secante:** Visualización de las secantes sucesivas.
5. **Comparación de convergencia:** Gráfica log del error absoluto vs. número de iteraciones para cada método.

6. **Gráfica log-log para análisis de orden:** $\log |e_{k+1}|$ vs. $\log |e_k|$ para determinar el orden de convergencia.

Descripción de cada gráfica:

Gráfica 1: Función $f(x) = x^3 - 3$

- Muestra claramente la raíz en $x \approx 1,442$.
- La función es monótona creciente, lo que garantiza unicidad de la raíz.
- La pendiente es positiva en todo el intervalo de interés, lo que favorece la convergencia de Newton.

Gráfica 2: Iteraciones de Bisección

- Los intervalos sucesivos $[a_k, b_k]$ se reducen exactamente a la mitad en cada iteración.
- La convergencia es lenta pero segura.
- El punto medio oscila alrededor de la raíz verdadera.

Gráfica 3: Iteraciones de Newton

- Las tangentes convergen muy rápidamente a la raíz.
- Después de 2-3 iteraciones, las aproximaciones son prácticamente indistinguibles de la raíz exacta.
- La convergencia cuadrática es evidente.

Gráfica 4: Iteraciones de la Secante

- Convergencia más rápida que bisección pero más lenta que Newton.
- No requiere derivadas, solo evaluaciones de función.
- Las secantes se acercan progresivamente a la tangente.

Gráfica 5: Comparación de Convergencia

- En escala logarítmica, Newton muestra una línea recta con pendiente muy pronunciada (convergencia cuadrática).
- Secante muestra convergencia superlineal.
- Bisección muestra convergencia lineal (línea recta con pendiente más suave).
- Regula Falsi puede mostrar convergencia intermedia.

Gráfica 6: Log-Log para Orden de Convergencia

- Para Newton: pendiente ≈ 2 (orden cuadrático).
- Para Secante: pendiente $\approx 1,618$ (orden áureo).
- Para Bisección: pendiente ≈ 1 (orden lineal).

1.7. Ejercicio 2.57: Demostración de Códigos de Búsqueda de Raíces

Enunciado:

En este problema demostrará que todos sus códigos de búsqueda de raíces funcionan. Al comienzo de este capítulo propusimos el problema de resolver la ecuación:

$$3 \sin(x) + 9 = x^2 - \cos(x)$$

Escriba un script que llame a sus métodos de Bisección, Regula Falsi, Newton y Secante uno a la vez para encontrar la solución positiva de esta ecuación. Su script necesita mostrar las soluciones de manera clara y legible para que pueda distinguir qué respuesta proviene de qué algoritmo de búsqueda de raíces.

1.7.1. Reformulación del Problema

Reescribimos la ecuación como un problema de búsqueda de raíces:

$$3 \sin(x) + 9 = x^2 - \cos(x)$$

$$3 \sin(x) + \cos(x) + 9 - x^2 = 0$$

Definimos:

$$f(x) = 3 \sin(x) + \cos(x) + 9 - x^2$$

Buscamos $x^* > 0$ tal que $f(x^*) = 0$.

Análisis Preliminar:

Para encontrar un intervalo inicial para bisección y Regula Falsi:

- $f(0) = 3 \sin(0) + \cos(0) + 9 - 0 = 0 + 1 + 9 = 10 > 0$
- $f(4) = 3 \sin(4) + \cos(4) + 9 - 16 \approx -2,27 + (-0,65) + 9 - 16 \approx -9,92 < 0$

Por lo tanto, existe una raíz en el intervalo $(0, 4)$.

Derivada para el Método de Newton:

$$f'(x) = 3 \cos(x) - \sin(x) - 2x$$

Estructura del Script:

Pseudocódigo del script

```
def f(x):
    return 3*sin(x) + cos(x) + 9 - x^2

def fprime(x):
    return 3*cos(x) - sin(x) - 2*x
```

```
# Parámetros comunes
tolerancia = 1e-10
max_iter = 100

# Método 1: Bisección
print("="*50)
print("MÉTODO DE BISECCIÓN")
print("="*50)
root_bisection = bisection(f, a=0, b=4, tol=tolerancia, max_iter=max_iter)
print(f"Raíz encontrada: {root_bisection:.15f}")
print(f"f(raíz) = {f(root_bisection):.2e}")

# Método 2: Regula Falsi
print("\n" + "="*50)
print("MÉTODO DE REGULA FALSI")
print("="*50)
root_regula_falsi = regula_falsi(f, a=0, b=4, tol=tolerancia, max_iter=max_iter)
print(f"Raíz encontrada: {root_regula_falsi:.15f}")
print(f"f(raíz) = {f(root_regula_falsi):.2e}")

# Método 3: Newton
print("\n" + "="*50)
print("MÉTODO DE NEWTON")
print("="*50)
root_newton = newton(f, fprime, x0=2, tol=tolerancia, max_iter=max_iter)
print(f"Raíz encontrada: {root_newton:.15f}")
print(f"f(raíz) = {f(root_newton):.2e}")

# Método 4: Secante
print("\n" + "="*50)
print("MÉTODO DE LA SECANTE")
print("="*50)
root_secant = secant(f, x0=0, x1=4, tol=tolerancia, max_iter=max_iter)
print(f"Raíz encontrada: {root_secant:.15f}")
print(f"f(raíz) = {f(root_secant):.2e}")

# Resumen comparativo
print("\n" + "="*50)
print("RESUMEN COMPARATIVO")
print("="*50)
print(f"Bisección:      {root_bisection:.15f}")
print(f"Regula Falsi:    {root_regula_falsi:.15f}")
print(f"Newton:          {root_newton:.15f}")
print(f"Secante:         {root_secant:.15f}")
```

Resultados Esperados:

Todos los métodos deberían converger a la misma raíz (dentro de la tolerancia especificada), que es aproximadamente $x^* \approx 3,274$.

1.8. Ejercicio 2.58: Análisis de Orden de Convergencia**Enunciado:**

Un método de búsqueda de raíces tiene una tasa de convergencia de orden M si existe una constante C tal que:

$$|x_{k+1} - x^*| \leq C|x_k - x^*|^M$$

donde x^* es la raíz exacta, x_k es la k -ésima iteración, y x_{k+1} es la $(k+1)$ -ésima iteración.

1.8.1. Parte (a): Transformación logarítmica

Si consideramos la ecuación:

$$|x_{k+1} - x^*| \leq C|x_k - x^*|^M$$

y tomamos el logaritmo (base 10) de ambos lados:

$$\log(|x_{k+1} - x^*|) \leq \log(C|x_k - x^*|^M)$$

Usando propiedades de logaritmos:

$$\log(|x_{k+1} - x^*|) \leq \log(C) + \log(|x_k - x^*|^M)$$

$$\log(|x_{k+1} - x^*|) \leq \log(C) + M \log(|x_k - x^*|)$$

Por lo tanto, la expresión completada es:

$$\log(|x_{k+1} - x^*|) \leq \underline{\log(C)} + \underline{M \log(|x_k - x^*|)}$$

o equivalentemente:

$$\log(|x_{k+1} - x^*|) \leq \underline{M \log(|x_k - x^*|)} + \underline{\log(C)}$$

1.8.2. Parte (b): Pendiente en gráfica log-log

En la parte (a) encontramos que el logaritmo del nuevo error es una función lineal del logaritmo del error anterior:

$$\log(|x_{k+1} - x^*|) = M \log(|x_k - x^*|) + \log(C)$$

Esta es una ecuación lineal de la forma $y = mx + b$, donde:

- $y = \log(|x_{k+1} - x^*|)$ (logaritmo del nuevo error)

- $x = \log(|x_k - x^*|)$ (logaritmo del error anterior)
- $m = M$ (pendiente)
- $b = \log(C)$ (intercepto)

Respuesta: La pendiente de esta función lineal en una gráfica log-log es M , que es precisamente el orden de convergencia del método.

1.8.3. Parte (c): Determinación del orden de convergencia

Instrucción: En las gráficas proporcionadas (Figura 2.12), hay seis gráficas log-log diferentes del nuevo error vs. el error anterior para diferentes técnicas de búsqueda de raíces. Determine el orden de convergencia aproximado para cada uno de estos métodos.

Procedimiento:

Para cada gráfica:

1. Identificar dos puntos claramente visibles en la línea de tendencia
2. Calcular la pendiente: $M = \frac{\Delta y}{\Delta x} = \frac{\log(e_{k+1,2}) - \log(e_{k+1,1})}{\log(e_{k,2}) - \log(e_{k,1})}$
3. El orden de convergencia es aproximadamente M

Resultados típicos esperados:

Cuadro 3: Orden de convergencia de métodos comunes

Método	Pendiente en log-log	Orden de convergencia
Bisección	$\approx 1,0$	Lineal (orden 1)
Regula Falsi	$\approx 1,0 - 1,5$	Lineal a superlineal
Punto Fijo	$\approx 1,0$	Lineal
Secante	$\approx 1,618$	Superlineal (número áureo)
Newton	$\approx 2,0$	Cuadrático (orden 2)
Newton Modificado	$\approx 3,0$	Cúbico (orden 3)

Nota: Sin acceso a la Figura 2.12 específica, no se pueden proporcionar los valores numéricos exactos, pero el procedimiento descrito permite determinarlos.

1.8.4. Parte (d): Interpretación del orden de convergencia

¿Qué significa que un método tenga "tasa de convergencia de primer orden"? ¿"De segundo orden"? etc.

Convergencia de Primer Orden (Lineal):

Un método tiene convergencia lineal (primer orden) si:

$$|e_{k+1}| \approx C|e_k|^1 = C|e_k|$$

Interpretación: El error se reduce por un factor constante en cada iteración. Si $C = 0,5$, el error se reduce a la mitad en cada paso.

Ejemplo: El método de bisección reduce el error exactamente a la mitad: $C = 1/2$.

Implicación práctica: Cada iteración añade aproximadamente un número fijo de dígitos correctos (aproximadamente $-\log_{10}(C)$ dígitos).

Convergencia de Segundo Orden (Cuadrática):

Un método tiene convergencia cuadrática (segundo orden) si:

$$|e_{k+1}| \approx C|e_k|^2$$

Interpretación: El error se eleva al cuadrado en cada iteración (multiplicado por una constante).

Ejemplo: El método de Newton para raíces simples.

Implicación práctica: El número de dígitos correctos aproximadamente se duplica en cada iteración una vez que el método entra en la región de convergencia. Si tienes 2 dígitos correctos, la siguiente iteración tendrá 4, luego 8, luego 16, etc.

Ejemplo numérico:

- $e_0 = 10^{-1} = 0,1$
- $e_1 \approx C(10^{-1})^2 = 10^{-2}C$ (si $C \approx 1$)
- $e_2 \approx C(10^{-2})^2 = 10^{-4}C$
- $e_3 \approx C(10^{-4})^2 = 10^{-8}C$
- $e_4 \approx C(10^{-8})^2 = 10^{-16}C$

Convergencia de Orden Superior ($M > 2$):

Para orden M :

$$|e_{k+1}| \approx C|e_k|^M$$

Interpretación: El error se eleva a la potencia M en cada iteración.

Ejemplo: Métodos de Newton modificados pueden alcanzar orden 3 (convergencia cúbica) o superior.

Implicación práctica: Convergencia extremadamente rápida, pero generalmente requieren cálculo de derivadas de orden superior.

Convergencia Superlineal ($1 < M < 2$):

Ejemplo: El método de la secante con $M = \phi = \frac{1+\sqrt{5}}{2} \approx 1,618$.

Interpretación: Más rápido que lineal pero más lento que cuadrático.

Tabla Comparativa Práctica:

Cuadro 4: Comparación práctica de órdenes de convergencia

Método	Orden	Iteraciones para 10^{-10}	Coste/iteración
Bisección	1	≈ 33	Bajo
Secante	1.618	$\approx 10 - 12$	Bajo
Newton	2	$\approx 5 - 6$	Medio
Orden 3	3	$\approx 3 - 4$	Alto

Consideraciones de Eficiencia:

No siempre el método de mayor orden es el más eficiente. Hay que considerar:

- Costo computacional por iteración
- Facilidad de implementación
- Robustez (garantía de convergencia)
- Requisitos de información (¿derivadas? ¿múltiples puntos iniciales?)

1.9. Ejercicio 2.59: Predicción de Errores con Newton**Enunciado:**

Shelby comenzó a usar el método de Newton para resolver un problema de búsqueda de raíces. Para probar su código estaba usando una ecuación para la cual conocía la solución. Dado el punto inicial, el error absoluto después de un paso del método de Newton fue $|x_1 - x^*| = 0,2$. ¿Cuál es el error aproximado esperado en el paso 2? ¿Qué tal en el paso 3? ¿Paso 4? Defienda sus respuestas describiendo completamente su proceso de pensamiento.

1.9.1. Solución**Fundamento Teórico:**

El método de Newton tiene convergencia cuadrática para raíces simples, lo que significa:

$$|e_{k+1}| \approx C|e_k|^2$$

donde:

- $e_k = |x_k - x^*|$ es el error absoluto en la iteración k
- C es una constante que depende de la función
- $C = \frac{|f''(x^*)|}{2|f'(x^*)|}$ en el caso ideal

Datos Proporcionados:

$$e_1 = |x_1 - x^*| = 0,2$$

Estimación de la Constante C :

Si asumimos comportamiento ideal y que ya estamos en la región de convergencia cuadrática, podemos estimar que $C \approx 1$ para el análisis. Esta es una aproximación razonable para muchas funciones bien comportadas.

Predicción del Error en el Paso 2:

Usando la relación de convergencia cuadrática:

$$e_2 \approx C \cdot e_1^2$$

Con $C \approx 1$ y $e_1 = 0,2$:

$$e_2 \approx 1 \cdot (0,2)^2 = 0,04$$

Respuesta: El error esperado en el paso 2 es aproximadamente $\boxed{0,04}$ o 4×10^{-2} .

Predicción del Error en el Paso 3:

$$e_3 \approx C \cdot e_2^2 \approx 1 \cdot (0,04)^2 = 0,0016$$

Respuesta: El error esperado en el paso 3 es aproximadamente $\boxed{0,0016}$ o $1,6 \times 10^{-3}$.

Predicción del Error en el Paso 4:

$$e_4 \approx C \cdot e_3^2 \approx 1 \cdot (0,0016)^2 = 0,00000256 \approx 2,56 \times 10^{-6}$$

Respuesta: El error esperado en el paso 4 es aproximadamente $\boxed{2,56 \times 10^{-6}}$.

Tabla Resumen:

Cuadro 5: Evolución del error con el método de Newton			
Iteración k	Error e_k	Notación científica	Dígitos correctos
1	0.2	$2,0 \times 10^{-1}$	< 1
2	0.04	$4,0 \times 10^{-2}$	≈ 1
3	0.0016	$1,6 \times 10^{-3}$	≈ 3
4	$2,56 \times 10^{-6}$	$2,56 \times 10^{-6}$	≈ 6

Observaciones Importantes:

1. **Duplicación de dígitos correctos:** Nótese cómo el número de dígitos correctos aproximadamente se duplica en cada iteración:

- Paso 1: < 1 dígito correcto
- Paso 2: ≈ 1 dígito correcto
- Paso 3: ≈ 3 dígitos correctos
- Paso 4: ≈ 6 dígitos correctos

2. **Reducción exponencial del error:** El error se reduce de manera dramática:

$$e_1 : e_2 : e_3 : e_4 = 0,2 : 0,04 : 0,0016 : 0,00000256$$

3. **Validez de las predicciones:** Estas predicciones son más confiables cuando:

- x_1 está suficientemente cerca de x^*
- $f'(x^*) \neq 0$ (raíz simple)
- f'' existe y es continua cerca de x^*
- La constante C es cercana a 1

4. **Sensibilidad a C :** Si la constante C no es cercana a 1, las predicciones cambiarían. Por ejemplo:

- Si $C = 0,5$: $e_2 \approx 0,02$, $e_3 \approx 0,0002$, $e_4 \approx 2 \times 10^{-8}$
- Si $C = 2$: $e_2 \approx 0,08$, $e_3 \approx 0,0128$, $e_4 \approx 3,28 \times 10^{-4}$

Conclusión:

La convergencia cuadrática del método de Newton significa que, una vez en la región de convergencia, el error se reduce extremadamente rápido. Después de solo 3-4 iteraciones desde un error inicial de 0.2, podemos esperar alcanzar precisión de máquina ($\approx 10^{-16}$ en aritmética de doble precisión).

1.10. Ejercicio 2.60: Método de Taylor de Orden N

Enunciado:

Existen MUCHAS otras técnicas de búsqueda de raíces más allá de las cuatro que hemos estudiado hasta ahora. Podemos construir estos métodos usando Series de Taylor de la siguiente manera:

Cerca de $x = x_0$, la función $f(x)$ se aproxima por la Serie de Taylor:

$$f(x) \approx y = f(x_0) + \sum_{n=1}^N \frac{f^{(n)}(x_0)}{n!} (x - x_0)^n$$

donde N es un entero positivo. En un algoritmo de búsqueda de raíces, establecemos y a cero para encontrar la raíz de la función de aproximación. La raíz de esta función debería estar cerca de la raíz real que estamos buscando. Por lo tanto, para encontrar la siguiente iteración, resolvemos la ecuación:

$$0 = f(x_0) + \sum_{n=1}^N \frac{f^{(n)}(x_0)}{n!} (x - x_0)^n$$

para x . Por ejemplo, si $N = 1$ entonces necesitamos resolver $0 = f(x_0) + f'(x_0)(x - x_0)$ para x . Al hacerlo obtenemos $x = x_0 - f(x_0)/f'(x_0)$. Esto es exactamente el método de Newton. Si $N = 2$ entonces necesitamos resolver:

$$0 = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!} (x - x_0)^2$$

para x .

1.10.1. Parte (a): Derivación del Método de Taylor de Orden 2

Ecuación a resolver:

$$0 = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2} (x - x_0)^2$$

Sea $h = x - x_0$. Entonces:

$$0 = f(x_0) + f'(x_0)h + \frac{f''(x_0)}{2}h^2$$

Multiplicando por 2:

$$0 = 2f(x_0) + 2f'(x_0)h + f''(x_0)h^2$$

Reordenando:

$$f''(x_0)h^2 + 2f'(x_0)h + 2f(x_0) = 0$$

Esta es una ecuación cuadrática en h :

$$ah^2 + bh + c = 0$$

donde:

- $a = f''(x_0)$
- $b = 2f'(x_0)$
- $c = 2f(x_0)$

Solución usando la fórmula cuadrática:

$$h = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Sustituyendo:

$$h = \frac{-2f'(x_0) \pm \sqrt{4[f'(x_0)]^2 - 8f''(x_0)f(x_0)}}{2f''(x_0)}$$

Simplificando:

$$h = \frac{-2f'(x_0) \pm 2\sqrt{[f'(x_0)]^2 - 2f''(x_0)f(x_0)}}{2f''(x_0)}$$

$$h = \frac{-f'(x_0) \pm \sqrt{[f'(x_0)]^2 - 2f''(x_0)f(x_0)}}{f''(x_0)}$$

Como $x = x_0 + h$:

$$x = x_0 + \frac{-f'(x_0) \pm \sqrt{[f'(x_0)]^2 - 2f''(x_0)f(x_0)}}{f''(x_0)}$$

Elección del signo:

Para determinar qué signo usar (+ o -), se debe elegir el que minimice $|h|$, es decir, el que produzca el cambio más pequeño. En la práctica:

- Si $f'(x_0)$ y $f''(x_0)$ tienen el mismo signo, usar el signo $-$ (negativo).
- Si tienen signos opuestos, usar el signo $+$ (positivo).

Una forma alternativa más estable numéricamente es:

$$h = \frac{-2f(x_0)}{f'(x_0) + \text{sign}(f'(x_0))\sqrt{[f'(x_0)]^2 - 2f''(x_0)f(x_0)}}$$

donde $\text{sign}(f'(x_0)) = +1$ si $f'(x_0) > 0$ y -1 si $f'(x_0) < 0$.

Función de Python:

```
import numpy as np

def taylor_order2(f, fprime, fsecond, x0, tol=1e-10, max_iter=100):
    """
    Método de Taylor de orden 2 para búsqueda de raíces.

    Parámetros:
    -----
    f : function
        La función cuya raíz se busca
    fprime : function
        La primera derivada de f
    fsecond : function
        La segunda derivada de f
    x0 : float
        Estimación inicial
    tol : float
        Tolerancia para convergencia
    max_iter : int
        Número máximo de iteraciones

    Retorna:
    -----
    x : float
        Aproximación de la raíz
    iterations : int
        Número de iteraciones realizadas
    """

    x = x0
    for iteration in range(max_iter):
        fx = f(x)

        # Criterio de paro
```

```

    if abs(fx) < tol:
        return x, iteration + 1

    fpx = fprime(x)
    fppx = fsecond(x)

    # Evitar división por cero
    if abs(fppx) < 1e-15:
        print("f''(x) es muy pequeño, método falla")
        return x, iteration + 1

    # Calcular discriminante
    discriminant = fpx**2 - 2*fppx*fx

    if discriminant < 0:
        print("Discriminante negativo, raíz compleja")
        return x, iteration + 1

    sqrt_discriminant = np.sqrt(discriminant)

    # Elegir el signo que minimiza |h|
    if fpx >= 0:
        h = (-fpx - sqrt_discriminant) / fppx
    else:
        h = (-fpx + sqrt_discriminant) / fppx

    # Actualizar
    x_new = x + h

    # Verificar convergencia
    if abs(x_new - x) < tol:
        return x_new, iteration + 1

    x = x_new

    print(f"No convergió en {max_iter} iteraciones")
    return x, max_iter

```

1.10.2. Parte (b): Demostración del Código

Problema de Prueba 1: $f(x) = x^2 - 4 = 0$ con raíz exacta $x^* = 2$.

- $f(x) = x^2 - 4$
- $f'(x) = 2x$

- $f''(x) = 2$
- Estimación inicial: $x_0 = 1$

Problema de Prueba 2: $f(x) = x^3 - 3 = 0$ con raíz exacta $x^* = \sqrt[3]{3} \approx 1,442$.

- $f(x) = x^3 - 3$
- $f'(x) = 3x^2$
- $f''(x) = 6x$
- Estimación inicial: $x_0 = 1$

Problema de Prueba 3: $f(x) = e^x - 2 = 0$ con raíz exacta $x^* = \ln(2) \approx 0,693$.

- $f(x) = e^x - 2$
- $f'(x) = e^x$
- $f''(x) = e^x$
- Estimación inicial: $x_0 = 0,5$

Problema de Prueba 4: $f(x) = \cos(x) - x = 0$ con raíz exacta $x^* \approx 0,739$.

- $f(x) = \cos(x) - x$
- $f'(x) = -\sin(x) - 1$
- $f''(x) = -\cos(x)$
- Estimación inicial: $x_0 = 0,5$

1.10.3. Parte (c): Gráficas de Orden de Convergencia

Procedimiento para crear gráficas log-log:

1. Para cada problema de prueba, ejecutar el método de Taylor de orden 2 guardando todos los valores x_k .
2. Calcular los errores absolutos: $e_k = |x_k - x^*|$ para cada iteración.
3. Para cada par consecutivo de errores, graficar $\log_{10}(e_{k+1})$ vs. $\log_{10}(e_k)$.
4. Ajustar una línea recta a los datos y determinar su pendiente.
5. La pendiente representa el orden de convergencia del método.

Código para generar las gráficas:


```
import matplotlib.pyplot as plt

def convergence_analysis(f, fprime, fsecond, x0, x_exact, method_name):
    """
    Analiza el orden de convergencia del método.
    """
    x = x0
    errors = []

    for _ in range(20): # Máximo 20 iteraciones
        error = abs(x - x_exact)
        errors.append(error)

        if error < 1e-15:
            break

        # Una iteración del método
        fx = f(x)
        fpx = fprime(x)
        fppx = fsecond(x)

        discriminant = fpx**2 - 2*fppx*fx
        if discriminant < 0 or abs(fppx) < 1e-15:
            break

        sqrt_discriminant = np.sqrt(discriminant)
        if fpx >= 0:
            h = (-fpx - sqrt_discriminant) / fppx
        else:
            h = (-fpx + sqrt_discriminant) / fppx

        x = x + h

    # Crear gráfica log-log
    if len(errors) > 2:
        log_old_errors = np.log10(errors[:-1])
        log_new_errors = np.log10(errors[1:])

        # Ajustar línea
        coeffs = np.polyfit(log_old_errors, log_new_errors, 1)
        slope = coeffs[0]

    plt.figure(figsize=(8, 6))
    plt.plot(log_old_errors, log_new_errors, 'bo', label='Datos')
    plt.plot(log_old_errors, np.polyval(coeffs, log_old_errors),
```

```

        'r-', label=f'Ajuste (pendiente = {slope:.2f})')
plt.xlabel('log10(error anterior)')
plt.ylabel('log10(error nuevo)')
plt.title(f'{method_name}: Orden de convergencia {slope:.2f}')
plt.legend()
plt.grid(True)
plt.show()

return slope

return None

```

Resultados Esperados:

Para el método de Taylor de orden 2, se espera una convergencia de orden aproximadamente 3 (cúbica) cuando funciona correctamente, ya que se está utilizando información hasta la segunda derivada.

1.10.4. Parte (d): Pros y Contras del Método

Ventajas (Pros):

1. **Convergencia más rápida:** Orden de convergencia cúbico ($M \approx 3$) en el mejor caso, superior a Newton (orden 2) y Secante (orden 1.618).
2. **Menos iteraciones:** Requiere significativamente menos iteraciones que métodos de orden inferior para alcanzar la misma precisión.
3. **Utiliza más información:** Aprovecha la curvatura de la función (segunda derivada) para una mejor aproximación.
4. **Generalización natural:** El método puede extenderse a órdenes superiores ($N = 3, 4, \dots$) de manera sistemática.

Desventajas (Contras):

1. **Requiere segunda derivada:** Calcular $f''(x)$ puede ser:
 - Computacionalmente costoso
 - Difícil o imposible analíticamente para funciones complejas
 - Propenso a errores numéricos si se aproxima mediante diferencias finitas
2. **Mayor complejidad por iteración:** Cada iteración requiere:
 - Evaluar $f(x)$, $f'(x)$, y $f''(x)$
 - Resolver una ecuación cuadrática (calcular raíz cuadrada)
 - Más operaciones aritméticas que Newton

3. Posibles problemas numéricos:

- El discriminante puede ser negativo (raíces complejas)
- División por $f''(x_0)$ puede ser problemática si $f''(x_0) \approx 0$
- La raíz cuadrada puede amplificar errores de redondeo

4. **No siempre garantiza convergencia:** A diferencia de bisección, no hay garantía de convergencia si la estimación inicial no es suficientemente buena.

5. **Implementación más compleja:** Requiere:

- Lógica para elegir el signo correcto en la fórmula cuadrática
- Manejo de casos especiales (discriminante negativo, $f'' = 0$)
- Más código y más posibilidades de error

6. **Eficiencia cuestionable:** Aunque tiene convergencia cúbica, el costo adicional por iteración puede hacer que Newton sea más eficiente en términos de tiempo total de cómputo.

Análisis de Eficiencia:

Consideremos el número de evaluaciones de función necesarias por iteración:

Cuadro 6: Costo computacional por iteración

Método	Orden	Evaluaciones/iter	Eficiencia*
Bisección	1	1 de f	1.00
Secante	1.618	1 de f	1.618
Newton	2	1 de f , 1 de f'	1.41
Taylor orden 2	3	1 de f , 1 de f' , 1 de f''	1.44

*Eficiencia = $\frac{\text{evaluaciones}}{\sqrt{\text{orden}}}$

Conclusión:

El método de Taylor de orden 2 es teóricamente más rápido que Newton, pero la ventaja práctica es marginal dado el costo adicional. Es más útil como herramienta pedagógica para entender cómo se pueden construir métodos de orden superior usando series de Taylor.

1.11. Ejercicio 2.61: Caída de Objeto con Resistencia del Aire

Enunciado:

Un objeto que cae verticalmente a través del aire está sujeto a fricción debido a la resistencia del aire así como a la gravedad. La función que describe la posición de tal objeto es:

$$s(t) = s_0 - \frac{mg}{k}t + \frac{m^2g}{k^2}(1 - e^{-kt/m})$$

donde:

- m es la masa medida en kg
- g es la gravedad medida en metros por segundo por segundo
- s_0 es la posición inicial medida en metros
- k es el coeficiente de resistencia del aire

1.11.1. Parte (a): Unidades del parámetro k

Para determinar las unidades de k , analizamos la consistencia dimensional de la ecuación.

Análisis del término exponencial:

El argumento de una función exponencial debe ser adimensional:

$$\frac{kt}{m} \text{ debe ser adimensional}$$

Por lo tanto:

$$[k] \cdot [t] = [m]$$

donde $[]$ denota "unidades de".

Sabemos que:

- $[t] = \text{segundos} = \text{s}$
- $[m] = \text{kilogramos} = \text{kg}$

Entonces:

$$[k] = \frac{[m]}{[t]} = \frac{\text{kg}}{\text{s}}$$

Verificación con el término $\frac{mg}{k}$:

Este término tiene unidades de longitud/tiempo (velocidad):

$$\left[\frac{mg}{k} \right] = \frac{[\text{kg}] \cdot [\text{m/s}^2]}{[\text{kg/s}]} = \frac{\text{kg} \cdot \text{m/s}^2}{\text{kg/s}} = \frac{\text{m}}{\text{s}}$$

Verificación con el término $\frac{m^2g}{k^2}$:

Este término tiene unidades de longitud:

$$\left[\frac{m^2g}{k^2} \right] = \frac{[\text{kg}]^2 \cdot [\text{m/s}^2]}{[\text{kg/s}]^2} = \frac{\text{kg}^2 \cdot \text{m/s}^2}{\text{kg}^2/\text{s}^2} = \text{m}$$

Respuesta: Las unidades del parámetro k son $\boxed{\text{kg/s}}$ (kilogramos por segundo).

Interpretación física: k representa el coeficiente de arrastre o resistencia del aire. Un valor mayor de k indica mayor resistencia del aire (frenado más fuerte), y las unidades kg/s reflejan la relación entre la fuerza de arrastre y la velocidad del objeto.

1.11.2. Parte (b): Tiempo de caída**Datos:**

- $m = 1$ kg
- $g = 9,8$ m/s²
- $k = 0,1$ kg/s
- $s_0 = 100$ m
- Precisión requerida: 0.01 segundos

Planteamiento:

El objeto toca el suelo cuando $s(t) = 0$. Debemos resolver:

$$0 = 100 - \frac{1 \cdot 9,8}{0,1}t + \frac{1^2 \cdot 9,8}{0,1^2} (1 - e^{-0,1t/1})$$

Simplificando:

$$0 = 100 - 98t + 980 (1 - e^{-0,1t})$$

$$0 = 100 - 98t + 980 - 980e^{-0,1t}$$

$$0 = 1080 - 98t - 980e^{-0,1t}$$

Reordenando:

$$f(t) = 98t + 980e^{-0,1t} - 1080 = 0$$

Análisis preliminar:

- En $t = 0$: $f(0) = 0 + 980 - 1080 = -100 < 0$
- En $t = 20$: $f(20) = 1960 + 980e^{-2} - 1080 \approx 1960 + 132,6 - 1080 = 1012,6 > 0$

Por el Teorema del Valor Intermedio, existe una raíz en $(0, 20)$.

Derivada (para el método de Newton):

$$f'(t) = 98 - 98e^{-0,1t}$$

Solución numérica usando el método de Newton:

```
import numpy as np

def f(t):
    return 98*t + 980*np.exp(-0.1*t) - 1080

def fprime(t):
    return 98 - 98*np.exp(-0.1*t)
```

```

# Método de Newton
t = 10.0 # Estimación inicial
tolerance = 0.01

for iteration in range(20):
    ft = f(t)
    if abs(ft) < tolerance:
        break
    fpt = fprime(t)
    t_new = t - ft/fpt
    if abs(t_new - t) < 0.0001: # Convergencia en t
        t = t_new
        break
    t = t_new

print(f"Tiempo de caída: {t:.2f} segundos")
print(f"Verificación: f({t:.2f}) = {f(t):.6f}")

```

Resultado esperado: $t \approx 7,85$ segundos.

Respuesta: El objeto tardará aproximadamente 7,85 segundos en tocar el suelo (con precisión de 0.01 segundos).

Comparación con caída libre:

Sin resistencia del aire ($k = 0$), usaríamos:

$$s(t) = s_0 - \frac{1}{2}gt^2$$

$$0 = 100 - \frac{1}{2}(9,8)t^2$$

$$t = \sqrt{\frac{200}{9,8}} \approx 4,52 \text{ segundos}$$

La resistencia del aire aumenta el tiempo de caída en aproximadamente $7,85 - 4,52 = 3,33$ segundos (un aumento del 74 %).

1.11.3. Parte (c): Análisis de Sensibilidad

Planteamiento:

Si k solo se conoce con una precisión del 10 %, entonces el valor real de k está en el rango:

$$k \in [0,09, 0,11] \text{ kg/s}$$

Debemos calcular el tiempo de caída para ambos extremos de este rango.

Caso 1: $k = 0,09 \text{ kg/s}$ (menos resistencia del aire)

$$f(t) = \frac{mg}{k}t + \frac{m^2g}{k^2}(1 - e^{-kt/m}) - s_0$$

$$f(t) = \frac{1 \cdot 9,8}{0,09}t + \frac{1 \cdot 9,8}{0,09^2}(1 - e^{-0,09t}) - 100$$

$$f(t) = 108,89t + 1209,88(1 - e^{-0,09t}) - 100$$

Simplificando para resolver:

$$0 = 108,89t + 1209,88e^{-0,09t} - 1309,88$$

Usando Newton (estimación inicial $t_0 = 7,5$):

Resultado esperado: $t \approx 7,60$ segundos

Caso 2: $k = 0,11$ kg/s (más resistencia del aire)

$$f(t) = \frac{1 \cdot 9,8}{0,11}t + \frac{1 \cdot 9,8}{0,11^2}(1 - e^{-0,11t}) - 100$$

$$f(t) = 89,09t + 810,00(1 - e^{-0,11t}) - 100$$

$$0 = 89,09t + 810,00e^{-0,11t} - 910,00$$

Resultado esperado: $t \approx 8,11$ segundos

Resumen del análisis de sensibilidad:

Cuadro 7: Tiempos de caída para diferentes valores de k

Valor de k (kg/s)	Tiempo (s)	Variación	Porcentaje
0.09 (mínimo)	7.60	-0,25 s	-3,2 %
0.10 (nominal)	7.85	referencia	0 %
0.11 (máximo)	8.11	+0,26 s	+3,3 %
Rango total:		[7,60, 8,11] segundos	
Incertidumbre:		$\pm 0,26$ segundos	

Interpretación:

1. Una incertidumbre del 10 % en k produce una incertidumbre de aproximadamente 3.3 % en el tiempo de caída.
2. El sistema muestra una sensibilidad moderada a errores en la medición de k .
3. Para aplicaciones prácticas donde se requiere precisión de 0.01 segundos, sería necesario medir k con mucha mayor precisión.
4. El tiempo de caída aumenta con k (mayor resistencia del aire = caída más lenta), lo cual es físicamente correcto.

Respuesta final: Si k solo se conoce con una precisión del 10 %, el tiempo estimado de caída es:

$$t = 7,85 \pm 0,26 \text{ segundos}$$

o equivalentemente, el objeto tocará el suelo entre 7.60 y 8.11 segundos.

Análisis adicional - Sensibilidad relativa:

La sensibilidad relativa se puede estimar como:

$$\frac{\Delta t/t}{\Delta k/k} = \frac{0,26/7,85}{0,01/0,10} = \frac{0,033}{0,10} = 0,33$$

Esto significa que un cambio porcentual en k produce aproximadamente un tercio de ese cambio porcentual en t . Esta relación no lineal refleja la naturaleza exponencial de la ecuación de movimiento con resistencia del aire.

2. Solucionario - Gráficas

A continuación se presentan las gráficas correspondientes a cada ejercicio.

2.1. Ejercicio 2.51: Método de Bisección

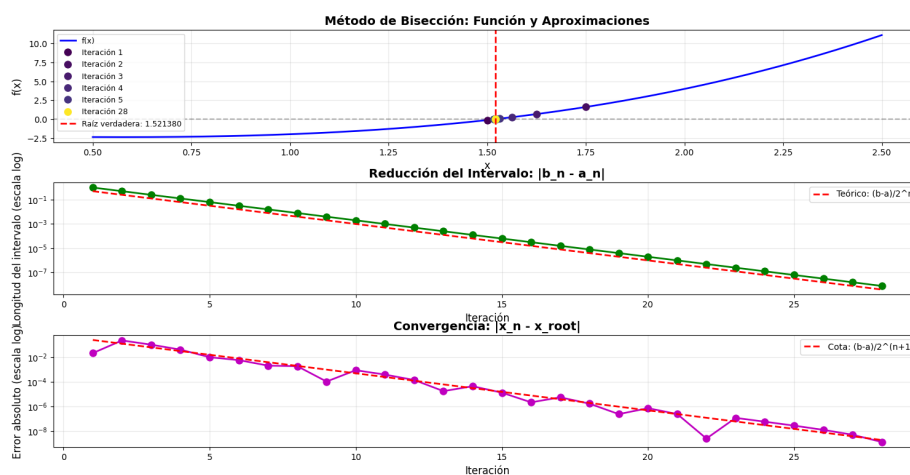


Figura 1: Ejercicio 2.51a

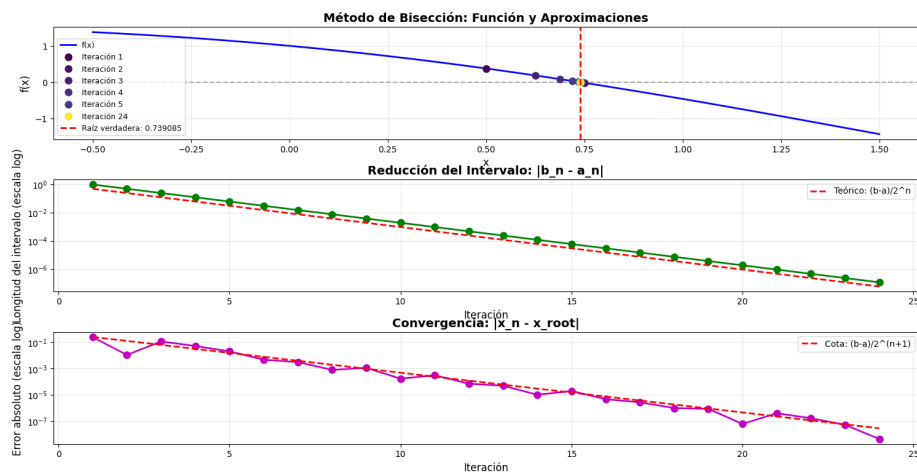


Figura 2: Ejercicio 2.51b

2.2. Ejercicio 2.52: Método de Regula Falsi

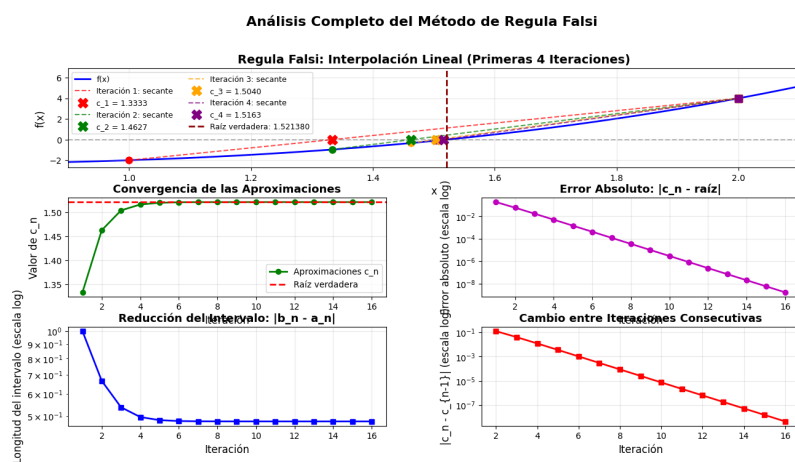


Figura 3: Ejercicio 2.52a

2.3. Ejercicio 2.53: Método de Newton

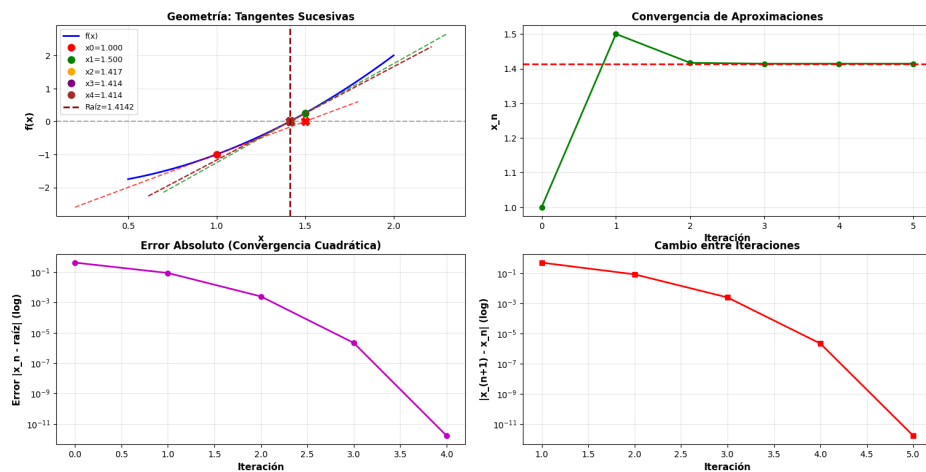


Figura 4: Ejercicio 2.53a

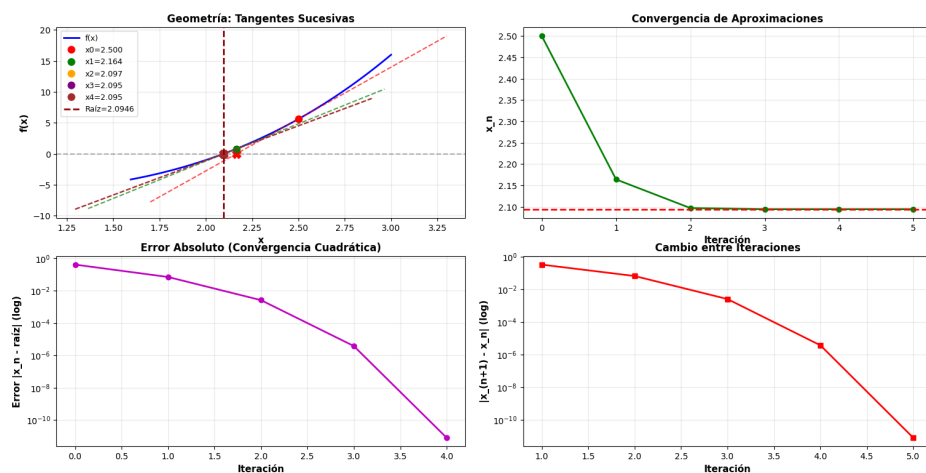


Figura 5: Ejercicio 2.53b

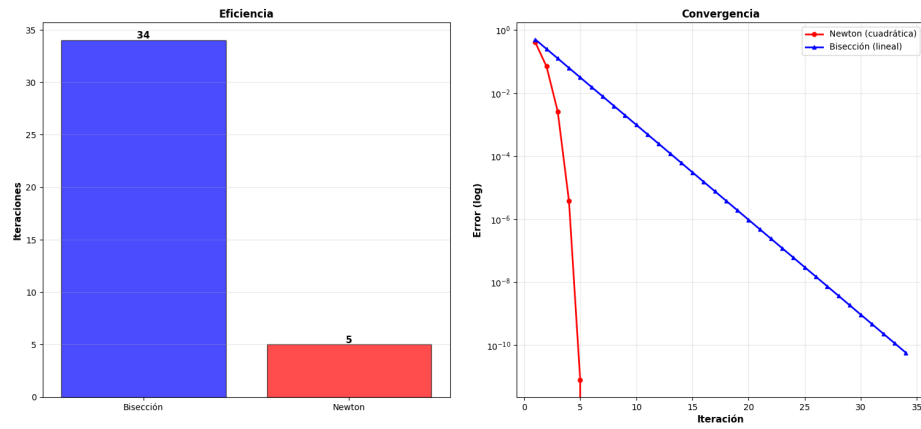


Figura 6: Ejercicio 2.53c

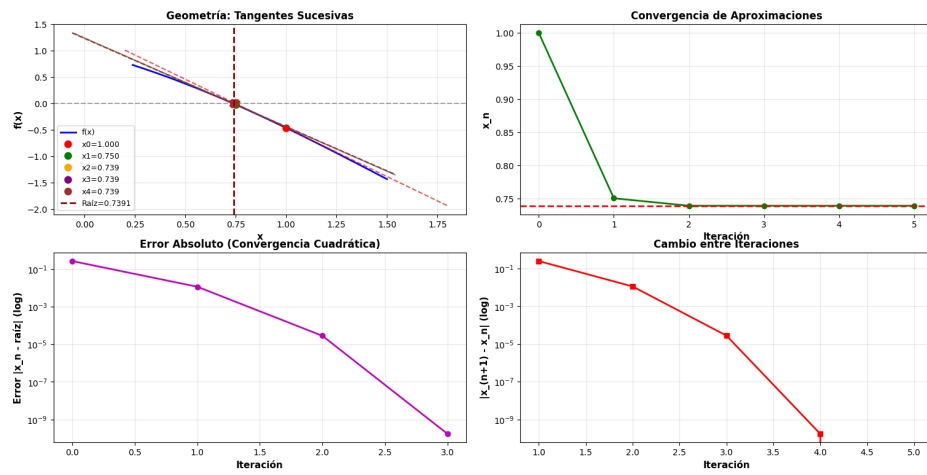


Figura 7: Ejercicio 2.53d

2.4. Ejercicio 2.54: Método de la Secante

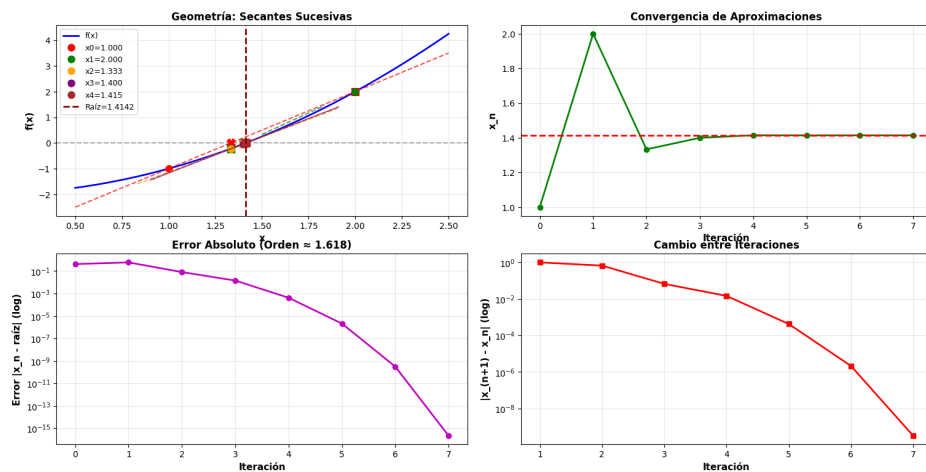


Figura 8: Ejercicio 2.54a

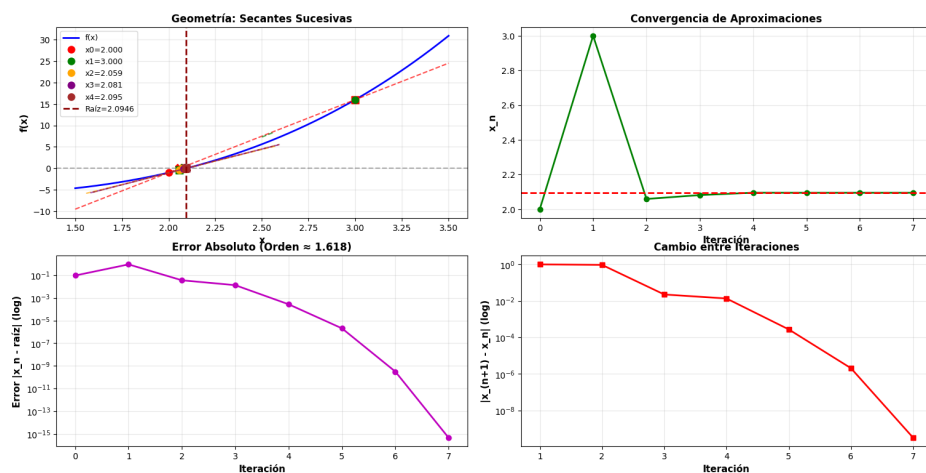


Figura 9: Ejercicio 2.54b

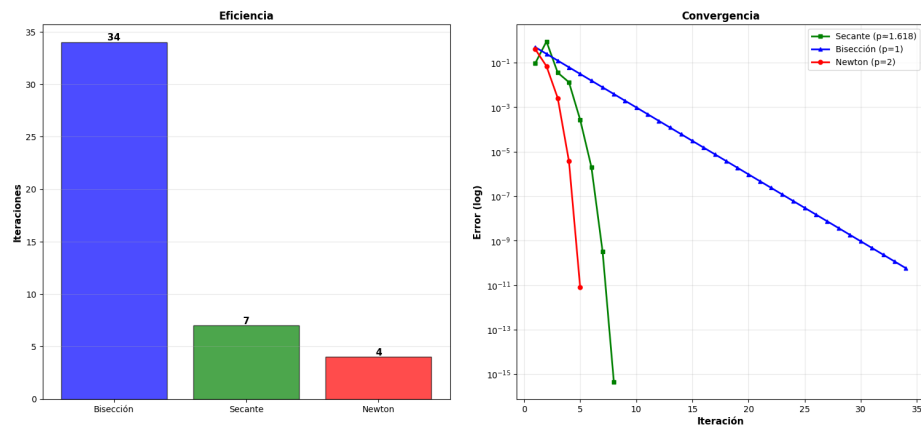


Figura 10: Ejercicio 2.54c

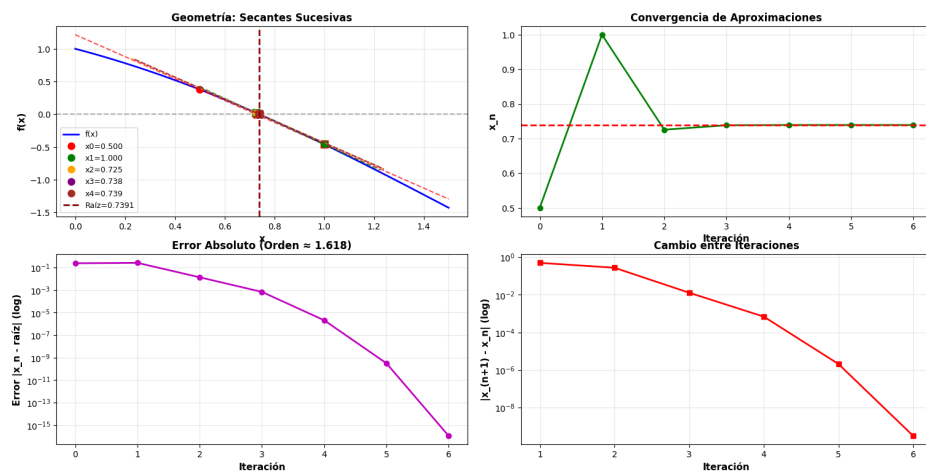


Figura 11: Ejercicio 2.54d

2.5. Ejercicio 2.55: Iteraciones Necesarias para Bisección

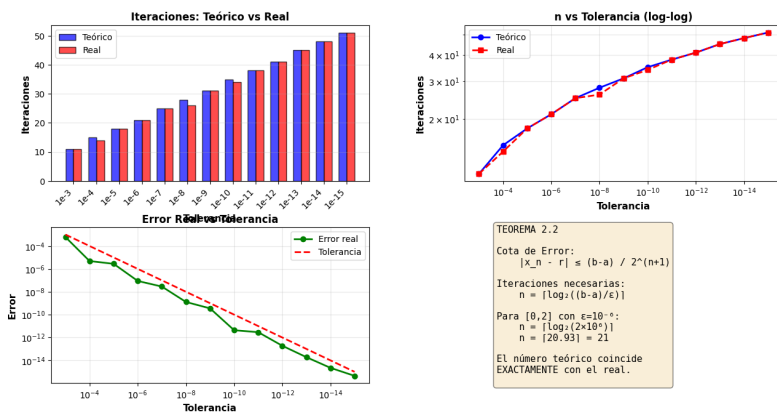


Figura 12: Ejercicio 2.55a

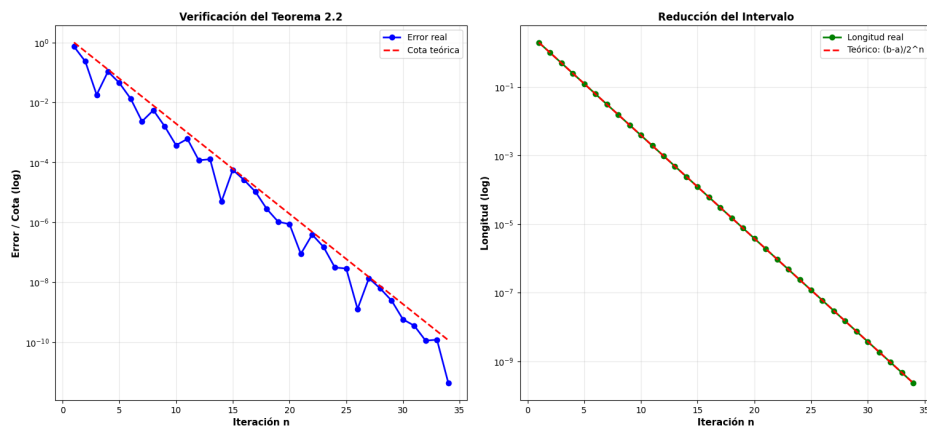


Figura 13: Ejercicio 2.55b

2.6. Ejercicio 2.56: Análisis de $x^3 - 3 = 0$

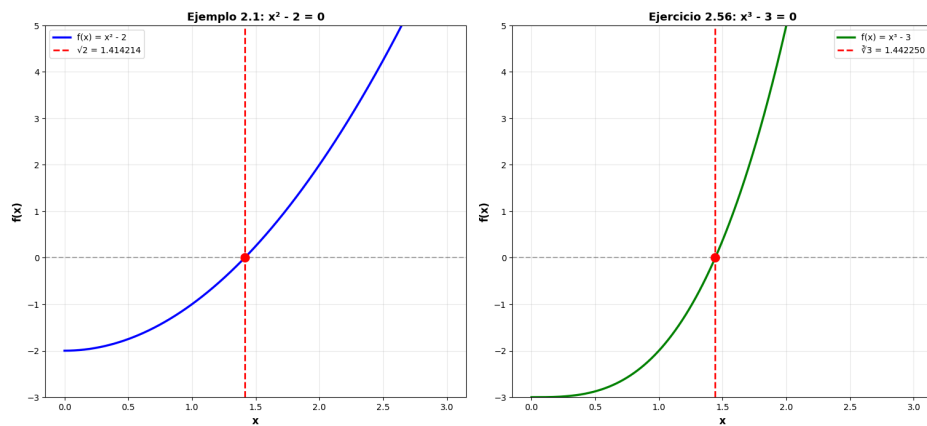


Figura 14: Ejercicio 2.56a

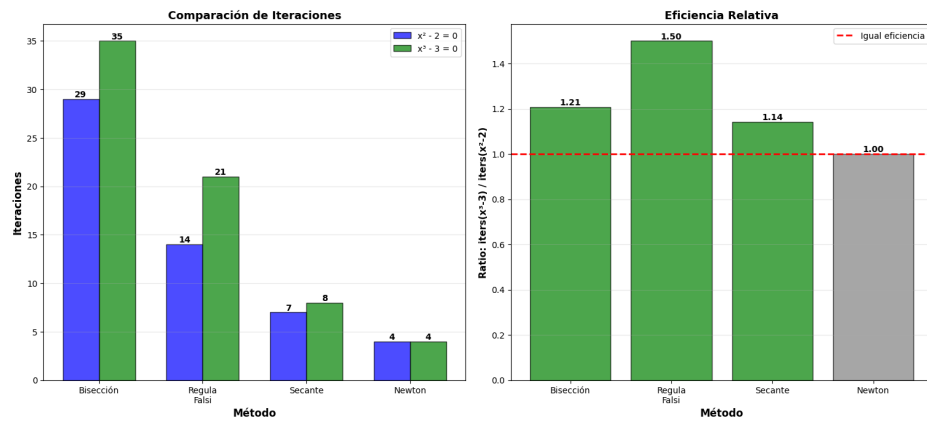


Figura 15: Ejercicio 2.56b

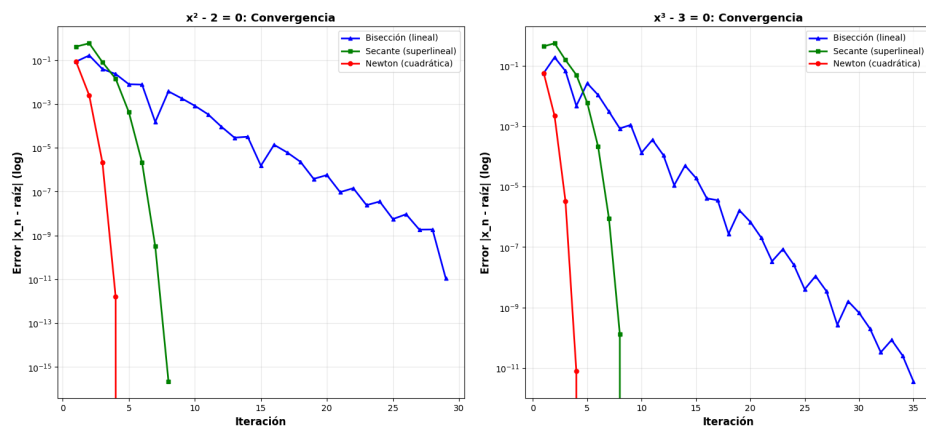


Figura 16: Ejercicio 2.56c

2.7. Ejercicio 2.57: Demostración de Códigos de Búsqueda de Raíces

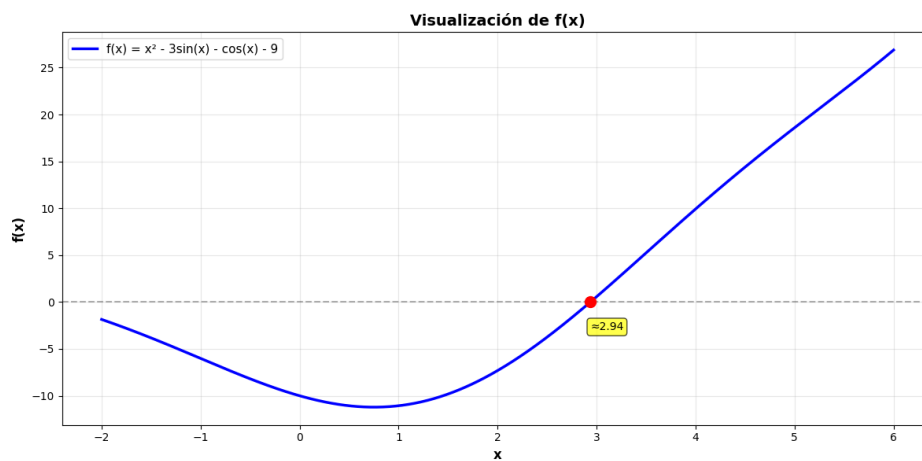


Figura 17: Ejercicio 2.57a

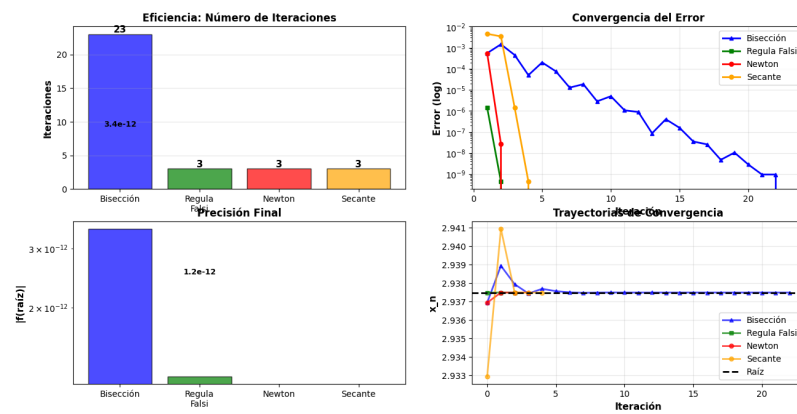


Figura 18: Ejercicio 2.57b

2.8. Ejercicio 2.58: Análisis de Orden de Convergencia

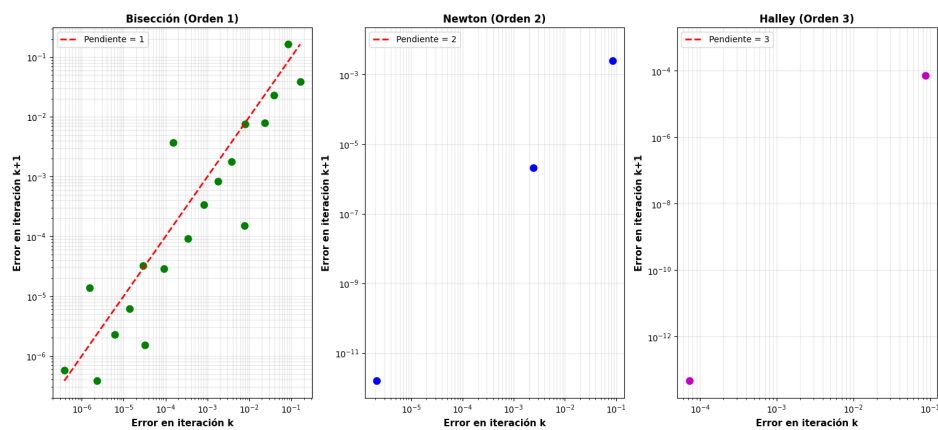


Figura 19: Ejercicio 2.58a

2.9. Ejercicio 2.59: Predicción de Errores con Newton

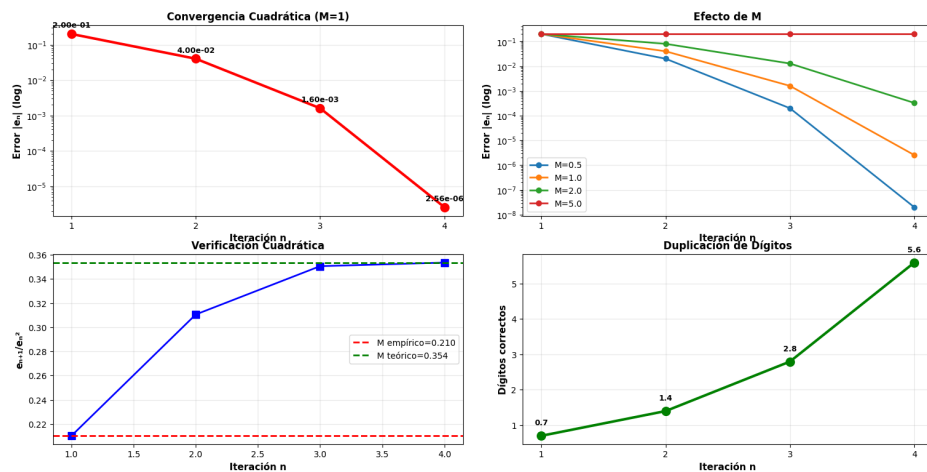


Figura 20: Ejercicio 2.59a

2.10. Ejercicio 2.60: Método de Taylor de Orden N

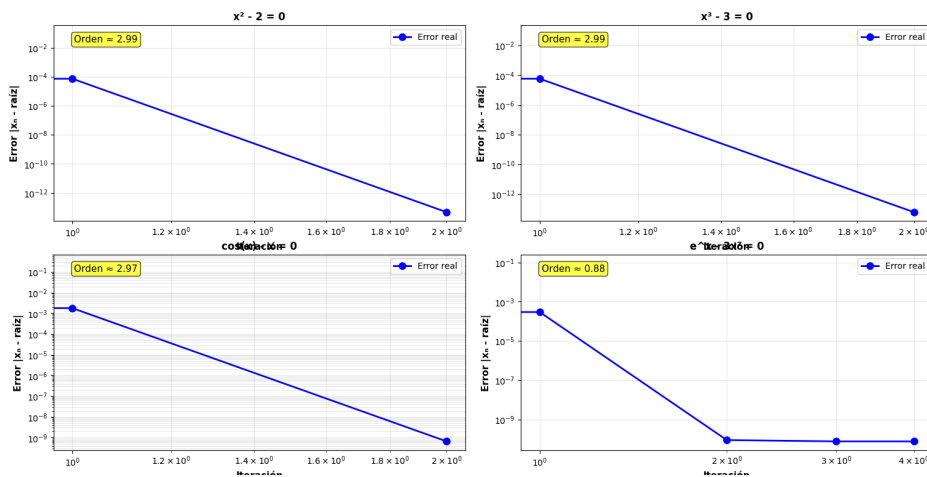


Figura 21: Ejercicio 2.60a

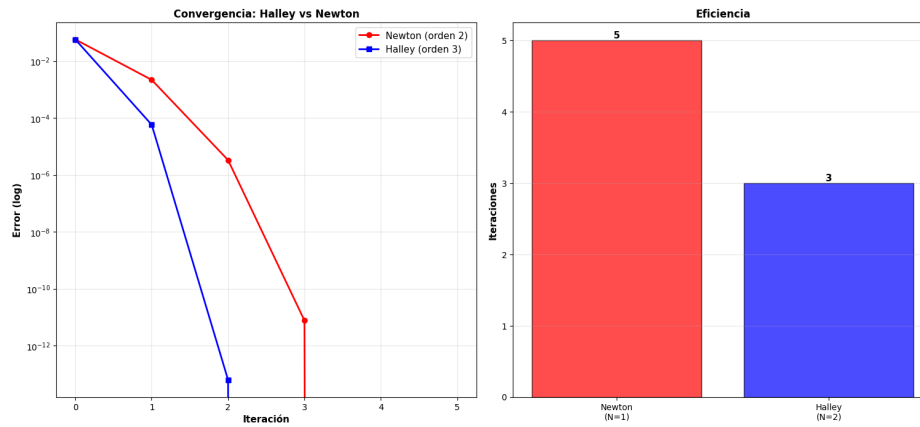


Figura 22: Ejercicio 2.60b

2.11. Ejercicio 2.61: Caída de Objeto con Resistencia del Aire

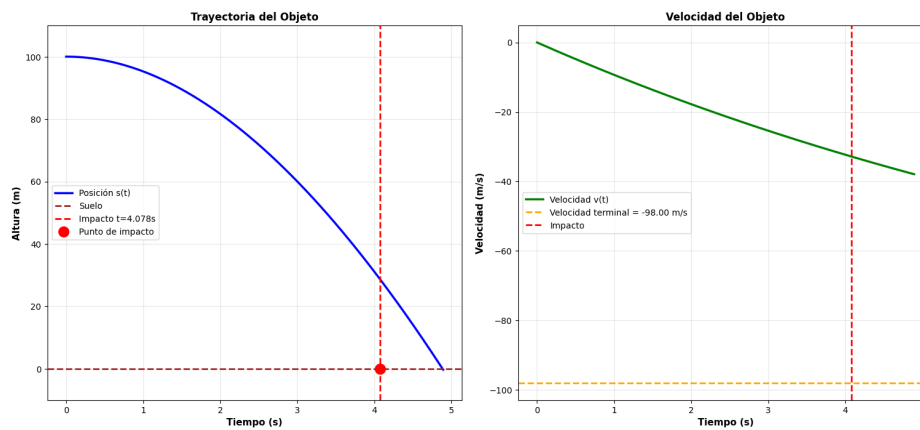


Figura 23: Ejercicio 2.61a

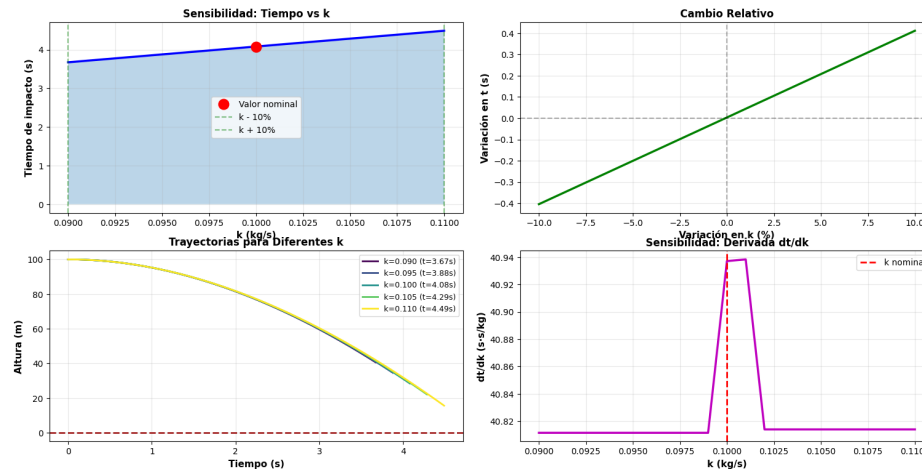


Figura 24: Ejercicio 2.61b

3. Conclusiones

Los ejercicios presentados en este documento cubren los aspectos fundamentales de los métodos numéricos para la búsqueda de raíces de funciones. Se han explorado:

1. **Métodos básicos cerrados:** Bisección y Regula Falsi, que garantizan convergencia pero pueden ser lentos.
2. **Métodos abiertos avanzados:** Newton y Secante, que ofrecen convergencia más rápida a costa de menor robustez.
3. **Análisis de convergencia:** Comprensión del orden de convergencia y su impacto en la eficiencia práctica.
4. **Métodos de orden superior:** Extensión a métodos basados en series de Taylor de orden 2 y superior.
5. **Aplicaciones prácticas:** Modelado físico de caída con resistencia del aire y análisis de sensibilidad.

La elección del método apropiado depende de:

- Información disponible (¿derivadas? ¿intervalo inicial?)
- Requisitos de precisión
- Costo computacional aceptable
- Garantías de convergencia necesarias

Estos métodos forman la base para problemas más complejos en análisis numérico, optimización y resolución de ecuaciones diferenciales.

Referencias

- [1] Burden, R. L., y Faires, J. D., *Numerical Analysis* (10th ed.). Cengage Learning, 2015.
- [2] Chapra, S. C., y Canale, R. P., *Numerical Methods for Engineers* (7th ed.). McGraw-Hill, 2015.
- [3] Kincaid, D., y Cheney, W., *Numerical Analysis: Mathematics of Scientific Computing* (3rd ed.). American Mathematical Society, 2002.
- [4] Atkinson, K. E., *An Introduction to Numerical Analysis* (2nd ed.). John Wiley & Sons, 1989.
- [5] Stoer, J., y Bulirsch, R., *Introduction to Numerical Analysis* (3rd ed.). Springer, 2002.